Coprocesadores : GPU, TPU y FPGA

Hermilo

17 de Febrero de 2026

Coprocesadores : ¿Por qué?

- Funcionalidad especializada.
  - Funciones no encontradas en CPU genérico.
  - Más rápidas que un CPU genérico.
- Diseño simplificado.
  - Menos costo que un CPU genérico.
  - No puede ejecutar sistema operativo.
  - Requiere de un CPU anfitrión.
- Más poder de cómputo a menor costo.
- Más compacto.
  - Menos espacio.
  - Menos energía.

Coprocesadores : Lo dificil.

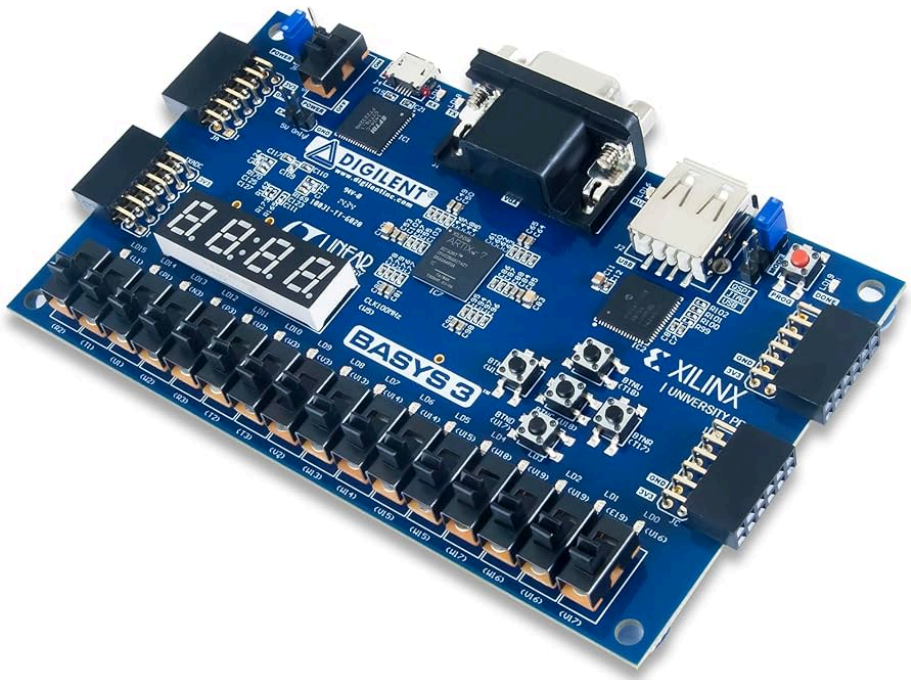
- Sistemas Heterogéneos.
  - CPU
  - Coprocesador.
- Detalles dependen del tipo de Coprocesador.
- Madurez de herramienta.
- Rendimiento práctico mucho menor que teórico.

Tipos de Coprocesadores

- Targetas de video : GPU.
  - Cómputo de alto rendimiento aprovecha mercado de video de alta calidad.
  - Entrenamiento Deep Learning.
- FPGA.
  - Circuitos programables.
  - Rápido y flexible.
  - Más lejos del diseño de un CPU genérico.

FPGA (Field Programmable Gate Array)

- Circuito integrado semiconductor reprogramable que permite configurar la lógica digital interna después de su fabricación
- Uso en electrónica programable.
- Diferentes fabricantes:
  - Altera
    - Intel compró Altera en 2015
  - Xylinx.
- Programación tipicamente en Verilog.
- Existen compiladores para HPC para C y Fortran.
- **Usado para resolver Modelos de Equilibrio General**



Programming FPGAs for Economics : An Introduction to Electrical Engineering Economics (Cheela et al., 2025)

QUANTITATIVE ECONOMICS
JOURNAL OF THE ECONOMETRIC SOCIETY

Original Articles | Open Access

Programming FPGAs for economics: An introduction to electrical engineering economics

Bhagath Cheela, André DeHon, Jesús Fernández-Villaverde, Alessandro Peri

First published: 03 February 2025 | https://doi.org/10.3982/QE2344 | Citations: 1

We thank Yicheng Li (UPenn, Engineering) for the initial implementation of our hardware design. We thank Lucas Ladenburger, Marina Leah McCann, and Paro Suh (CU Boulder, Economics) for outstanding research assistance. We thank Andrew Monaghan and the CU Boulder Research Computing Center for providing valuable insights. We also thank Giuseppe Bruno and Riccardo Russo (Bank of Italy) for testing an early version of the tutorial associated with this paper and Victor Duarte, Mahdi E. Kahou, and Jesse Perla for their comments. This project used: (i) the RMACC Summit supercomputer, supported by the National Science Foundation (awards ACI-1532235 and ACI-1532236), CU Boulder and Colorado State University; (ii) AWS Credits awarded under the NSF CC* Hybrid Cloud Award OAC-1925766, Research Computing, CU Boulder, 2022. This project was also supported by the Undergraduate Research Experiences for Diversity Grant, 2021, Institute of Behavioral Science, University of Colorado, USA.

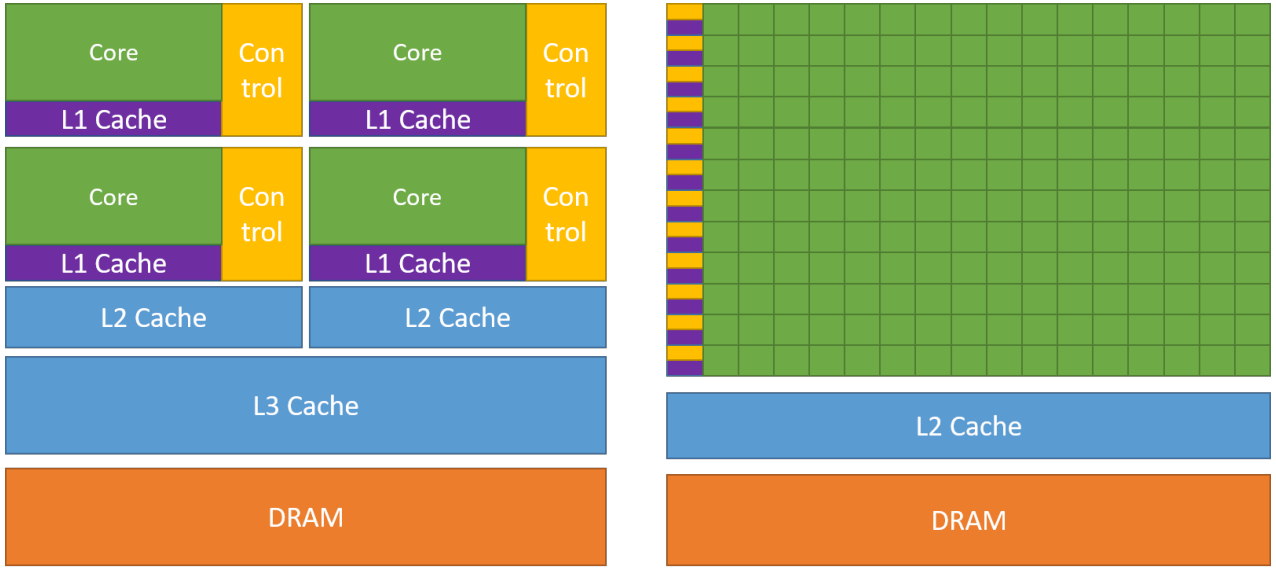
GPU (Graphics processing unit)

- Mercado dominado por NVIDIA.
- Arquitecturas : Tesla, Fermi, Kepler, Maxwell, Pascal, Volta, Turing, etc.
- **CUDA** : introducida 2007, con tesla.
- Parecido a C.
- Es posible programar GPU con **OpenMP** siguiendo el modelo fork-join.
- Problema : Portabilidad.
- **Mojo**!!

Mojo is blazing fast

- **100-1000X faster** than Python.
- Full power of modern CPUs.
  - SIMD, threads, NUMA, etc.
  - New MLIR-based compiler.
- Works on GPU too.
  - **Same Language** for both CPU and GPU.
- Power of Pythonic Mojo code.
  - Easy to learn.
  - Great for low level performance.
  - Powerful metaprogramming.

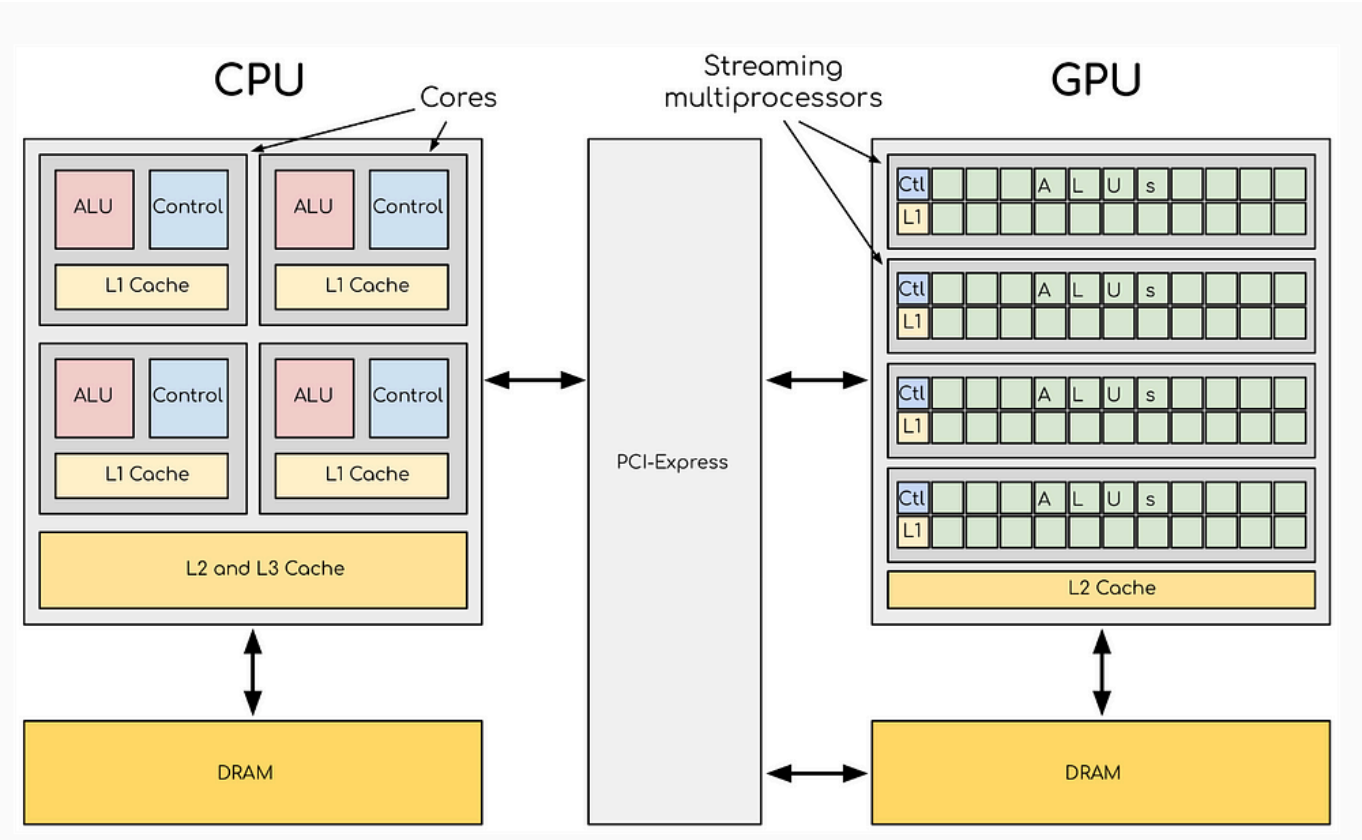
CPU, diseño orientado a latencia, GPU, diseño orientado a throughput[1].



CPU GPU

[1] The NVIDIA H100, for example, can run 16,896 threads simultaneously in a single clock cycle, with over 270,000 threads queued for execution.

Cómputo Heterogéneo



Conceptos clave

- Kernel.
- Thread / Block / Grid
- Host / Device Memory
- Global / Shared Memory

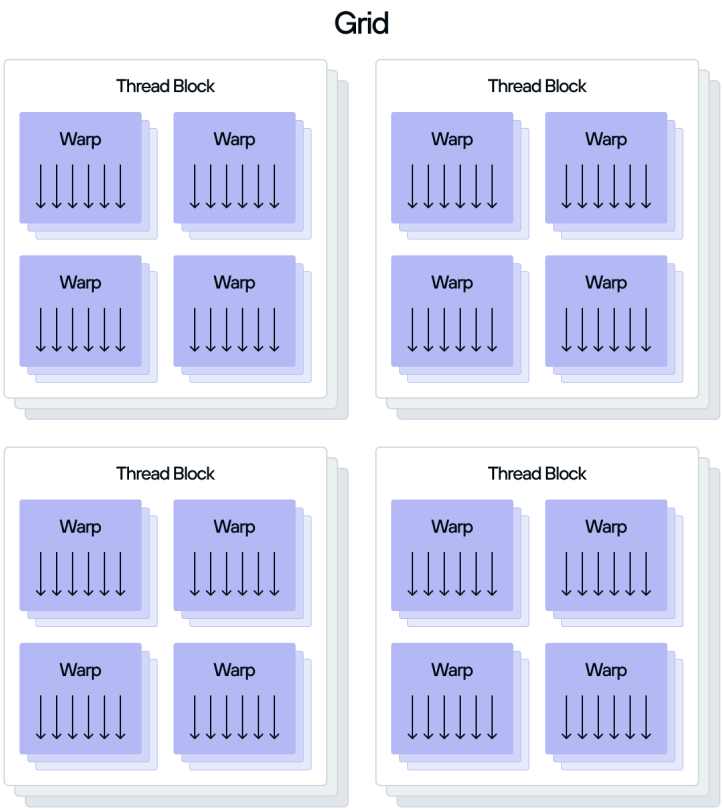
Kernel

Kernel function o GPU Kernel es simplemente una función que se ejecuta en la GPU, ejecutando un cálculo específico en un conjunto grande de datos en paralelo mediante miles o millones de threads.

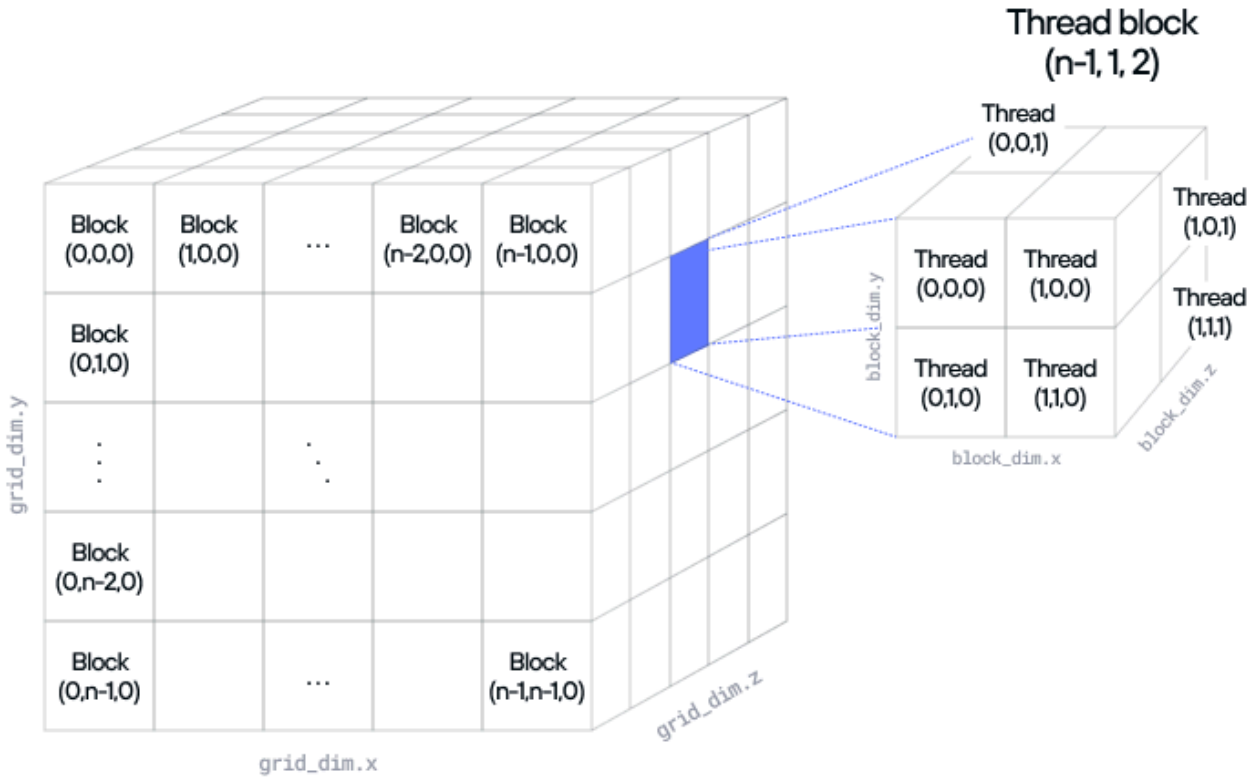
```python
from std.gpu import block_dim, block_idx, thread_idx
from std.math import ceildiv
# Calculate the number of thread blocks needed by dividing the vector size
# by the block size and rounding up.
comptime block_size = 256
comptime num_blocks = ceildiv(vector_size, block_size)


def vector_addition(
    lhs_tensor: LayoutTensor[float_dtype, layout, MutAnyOrigin],
    rhs_tensor: LayoutTensor[float_dtype, layout, MutAnyOrigin],
    out_tensor: LayoutTensor[float_dtype, layout, MutAnyOrigin],
):
    """Calculate the element-wise sum of two vectors on the GPU."""

    # Calculate the index of the vector element for the thread to process
    var tid = block_idx.x * block_dim.x + thread_idx.x

    # Don't process out of bounds elements
    if tid < vector_size:
        out_tensor[tid] = lhs_tensor[tid] + rhs_tensor[tid]
```

Thread

- Unidad básica de una ejecución paralela en una GPU.
- Cada thread corre el mismo kernel function pero procesa diferentes piezas de datos.
- Un **thread block** es un grupo cordinado de threads que pueden compartir memoria y sincronizarse entre sí.
- Thread dentro de un bloque pueden comunicarse muy eficientemente.
- Un **grid** es el nivel top de la estructura organizacional para la ejecución del kernel function.
- Un grid consiste de múltiples bloques de threads que se dividen aún más en threads individuales que ejecutan el kernel function concurrentemente.



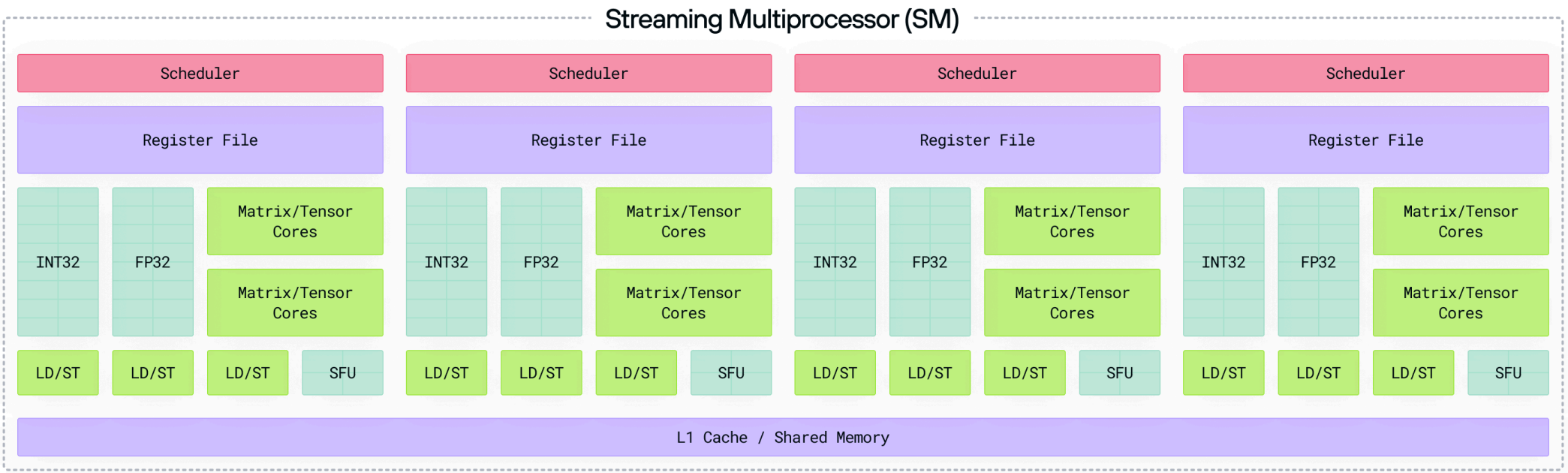
Organization of thread blocks and threads within a grid.



Global Memory y Shared Memory

- **Global Memory** es el espacio de memoria principal de la GPU accesible para todos los threads.
- El grande, pero relativamente lenta.
- **Shared Memory** es una memoria rápida onchip que los thread en un bloque usan para comunicarse.
- Es varios órdenes de magnitud más rápida que la memoria global, lo que la hace crítica para el desempeño.

Shared Memory

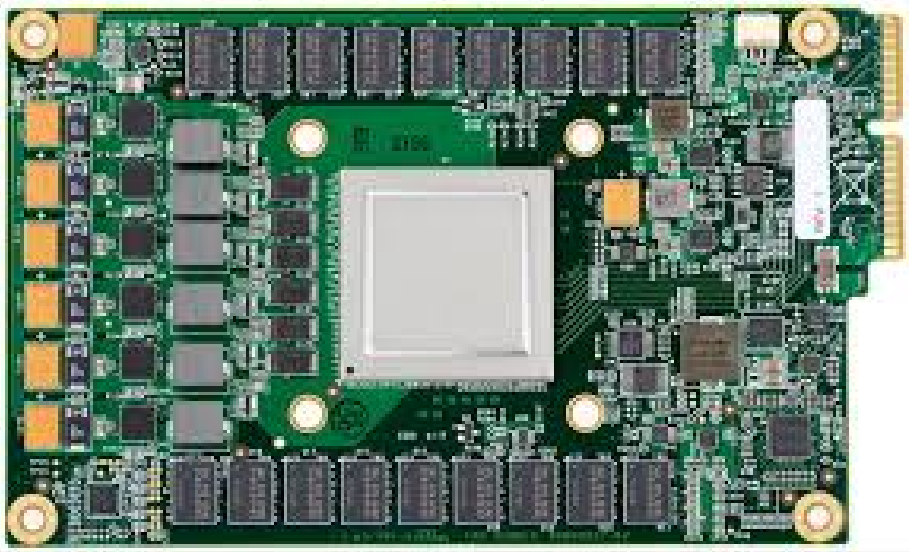


Aceleradores para Machine Learning y Deep Learning

- Intensivos en Álgebra Lineal.
- Productos para:
  - Equipos portátiles.
  - Sistemas embebidos (e.g coches)

TPU (Google Tensor Processing Unit)

- Para acelerar TensorFlow y **JAX**
- Acelerar el proceso de Entrenamiento.



JAX

Bibliography

Cheela, B., DeHon, A., Fernández-Villaverde, J., & Peri, A. (2025). Programming FPGAs for economics: An introduction to electrical engineering economics. *Quantitative Economics*, *16*(1), 49–87.